

把本体知识编译进图查询 Rewriting Ontology-Mediated Property Graph Queries into GQL

本篇范围说明：这是一篇 CIKM 2026 会议论文（arXiv 2608.20092v1，2026-08-20，萨尔茨堡大学 / 帕德博恩大学 / 维也纳工业大学）。下面「全文翻译」一节覆盖摘要、引言（含两个历史障碍与三条贡献）、改写算法的两个阶段的直觉说明、实验设置与结果、以及结论与未来工作。**描述逻辑的形式化定义、七条典范模型构造规则、五条转移添加规则的符号细节、以及全部引理与定理的证明，不逐条译——它们是正确性论证，而这份精读要传达的是「这套做法是什么、能不能用」。**

核心内容

Q：这篇论文解决的是什么问题？

A：一句话——**你有一张图数据库，还有一份本体（描述这个领域里的概念之间怎么包含、怎么推导）。现在有人来查询，查询里用到了本体里的词，而数据库根本不知道本体存在。怎么让他拿到完整答案？**

举个具体的例子说明「完整」的难点在哪。假设本体里写着「地铁站是一种交通站点」，而数据库里只把某个节点标成了「地铁站」。有人查「所有交通站点」——**数据库照字面查会返回空，因为没有有一个节点带着「交通站点」这个标签。**那个事实是被本体推导出来的，不在库里。

Q：有两条路可以走，这篇选了哪条？

A：这一段是理解全篇的关键。

做法	怎么做	代价
物化（materialise）	把本体能推出的所有隐含事实 都算出来写进库里	库要膨胀；本体一改就得全部重算；有些本体推出来的东西是无穷的
查询改写（query rewriting）	库不动，把本体的推理编译进查询本身—— 改写后的查询只用库里真实存在的词	查询会变复杂、变慢

这篇走的是第二条，而且是「纯改写」（pure rewriting）——论文自己下的定义是：**本体推理被完全编译进改写后的查询，不需要任何数据预处理，改写结果可以直接交给现成的查询引擎执行。**

这条路的好处论文在引言第一句就说了：**在不物化所有隐含事实的前提下，用领域知识扩展已有数据，同时仍然能查到全部推论。**

Q：这件事在关系数据库上早就有商用系统了，为什么图数据库上一直没有？

A：论文点出**两个历史障碍**，而且指出它们是同时消失的。

第一个障碍是查询语言不够用。 图查询的看家本领是**导航**——沿着边走任意长的路径（「从这个人出发，顺着『同事』这条边走任意多步，能到谁」）。这类查询的理论叫**正则路径查询**（regular path query, RPQ），因为路径的形状可以像正则表达式那样写（连接、并、星号）。而以前的实用图查询语言**只部分支持 RPQ**，还不支持这个场景里天然需要的同态语义。

第二个障碍更硬，是一个否定性结果：已经证明了，哪怕是最简单的 DL-Lite 本体，普通的 RPQ 也改写不成**合取正则路径查询**。要改写就必须用**嵌套的 RPQ**——而以前的技术不支持嵌套。

Q：那什么变了？

A：**GQL 与 SQL/PGQ 的 ISO 标准化。** 这两个新标准共享同一个导航内核，**内含嵌套双向正则路径查询与基本同态语义**，而实用图查询语言正在快速跟上。

两个障碍一次性被同一件事移除了——所以这篇论文的时机不是偶然：它是标准落地之后第一个能做的工作。

论文里有一句话把过去十年的状况说得很直白：

这些研究关注的是可判定性与计算复杂度的边界。**所提出的算法如此不适合实现，以致十多年过去，一个实用实现都没有出现过。**

Q：算法叫什么，长什么样？

A：叫 **ARCO**（Automata-based Rewriting of CN2RPQs under Ontologies，本体下合取嵌套双向正则路径查询的自动机式改写）。输入是一份 OWL 格式的本体加一个导航查询，输出是一句 Cypher（Neo4j 的查询语言）。

核心机制是两个「跳过」函数，理解了这两个就理解了整个算法。

先要有一个概念：典范模型（canonical model）。 它是「把本体的推理全部展开之后的那张图」——既包含库里原有的事实（论文叫**显式事实**），也包含推出来的（**隐式事实**）。改写的目标就是：让查询在**只有显式事实**的真实库上跑，却得到它在典范模型上会得到答案。

典范模型分成两部分，两个跳过函数各管一部分：

部分	是什么	对应函数
匿名部分	本体里写了「每个 A 都必然有一条 r 边指向某个东西」，于是 要凭空造出一个节点来满足它 。这些造出来的节点库里没有，叫匿名节点	skipAnon
核心	典范模型限制在库里真实存在的节点上	skipCore

skipAnon 处理的那个形状值得单独说，因为它是最反直觉的一段。 匿名节点是凭空造的，但它们**不是任意乱造**——论文证明了：如果库里有一个节点满足概念 C，那么由它长出来的所有匿名节点**构成一棵以它为根的树，而且这棵树在改名之内是唯一的**。既然唯一，就可以**预先算出来**：对每一个概念 C，算清楚「查询的哪一段可以完全在 C 长出来的那棵树里走完」。算清之后，遇到满足 C 的节点就**整段跳过**那部分查询。

而这个「预先算出来」的计算，被归约成了标准的描述逻辑推理——论文把查询自动机的每个状态变成一个新造的概念名，把自动机的转移变成概念包含公理，然后交给现成的推理器（实现里用的是 Hermit）判定。这是全篇最漂亮的一步：把一个关于自动机的问题翻译成一个本体推理问题，因为后者有成熟工具。

skipCore 的机制简单些：库里每一条隐式事实，都是由另一条事实触发的，顺着链子往回追总能追到一条显式事实。于是给查询自动机**添加转移**，允许它在找某条隐式事实时，改去找触发链上任何一条更靠前的事实。

Q：为什么要限制成「join-on-free」？

A：完整的合取查询（多个路径条件拼在一起、共享变量）改写是能做的，理论算法十多年前就有。这篇限制到 join-on-free（**非答案变量不能被多个原子共用**）是为了让算法保持实用。

论文对这个限制的辩护值得记下来，因为它是一个判断而不是一个证明：

我们预期，那些需要不同的正则路径分别行进、然后在典范模型的无名部分某处汇合的真实查询，**实践中极少出现，甚至根本不会出现。**

而且论文说了：完整版的算法他们在扩展版里做了，**但改写时间急剧上升，而增加的价值看起来有限，因为还没找到一个超出这个片段的好用例。**

Q：实验结果怎么样？

A：**这里有一个必须看清的分界：改写快，而执行慢。**

两份真实本体：

本体	概念数	角色数	Neo4j 节点数	关系数
DBPedia（维基百科抽取的数据）	1230	1212	1007.5 万	2219.4 万
MMM（蒙彼利埃都会区的下水道系统开放数据）	53	18	10.76 万	37.9 万

改写这一步是快的：DBPedia 上普遍在 250 毫秒以内，MMM 上约 600 毫秒。**注意这个反直觉的结果——概念和角色多得多的 DBPedia 反而更快。**原因是 DBPedia 里没有任何「右侧带存在量词」的公理，也就是不会造匿名节点，**skipAnon** 那一整套预计算不需要跑。所以真正决定改写耗时的不是本体规模，**而是本体里有没有那种会造匿名节点的公理。**

而执行这一步是慢的，这是论文自己承认的短板。DBPedia 上 100 条合成查询：要么 2 到 8 秒跑完，要么很快撞上 20 秒超时——**100 条里 56 条超时。**MMM 上手写的 5 条查询，3 条撞上超时（那里的上限是 1000 秒）。

论文对此给的是诚实的「不知道」：可能的原因是查询构造触发了笛卡尔积，也可能是答案数量太多；而**优化改写后查询的执行不在本文范围内。**它明确写下需要弄清究竟为什么会超时。

还有两处限制要一并记住，否则会高估这个结果：其一，100 条合成查询里有 62 条因为状态数太多（ ≥ 30 ）算不出幂集；其二，那些合成查询本来就不返回任何答案。

背景补充 / 点评

先把几个术语落到地面上，否则上面那些「导航」「嵌套」都是空的。

属性图 (labelled property graph, LPG) 就是 Neo4j 那种模型：节点和有向边，两者都能带多个**标签**（用来分类）和**属性**（键值对）。**RDF** 是另一套模型（三元组），本体那一派传统上长在 RDF 上。这篇的一个实质贡献是把本体那一套搬到 LPG 这边来——而且**专门支持属性值**，论文点出这是 LPG 的核心特性而**本体查询文献常常无视它**。

DL-Lite 是描述逻辑家族里刻意做得很轻的一支，轻的目的就是为了让查询改写可行（表达力再强一点，改写就不成立了）。这篇定义了一个变体 **DL-Lite[^]PG**，加的能力是**允许在公理左侧做属性值测试**——也就是可以写出「属性 p 大于等于 10 的那些节点构成一个概念」这样的东西。这一条对实际建模很有用，因为真实数据里的分类经常是按数值划的。

「**本体中介查询**」这个框架的老名字是 **OBDA** (ontology-based data access，基于本体的数据访问)。关系数据库上已有商用系统，图数据库上一一直空缺——这篇填的就是这个缺口。

再说这篇对我手上工作最直接的一处，它不是算法，是那个「物化 vs 改写」的分岔。

prove 现在的形态本质上是**编译**：把用户的问题编译成对数据仓库的查询，指标口径这类知识写在配置里、编译时被展开进 SQL。这在结构上就是「**查询改写**」那一路，不是「**物化**」那一路——我们没有为每个指标口径预先算一张物化表，而是每次把口径展开进查询。

而这篇论文给了这条路两个已知代价，都值得对照自查：

第一个代价是改写后的查询会变得难以执行。 论文里 100 条查询 56 条超时，而改写本身只用了 250 毫秒。也就是说「编译很快、执行很慢」这个失衡不是我们的特殊问题，是这条路的固有形状。prove 那边如果出现「口径复杂的指标查询特别慢」，第一反应不该是怀疑编译器写坏了，而是**先问改写后的查询形状是不是触发了执行引擎的坏路径**。论文点的两个可能原因（笛卡尔积、答案数太多）就是现成的排查方向。

第二个代价是限制的正当性只能靠经验判断，不能靠证明。 join-on-free 这个限制，论文的辩护是「实践中极少出现，甚至根本不会出现」——这是一句**经验判断**，论文自己也是这么标的。它同时给了不做完整版的真实理由：完整版做了，但改写时间急剧上升而收益不明。这个形状我们也有：prove 支持的查询形态一直是有边界的，而边界的正当性来自「用户实际上不这么问」。这类正当性会随用户变化而失效，所以它**需要被复查，而不是当成定理**。论文的做法是把限制和它的理由都写在纸面上，这一点值得学。

第三处，那个把自动机问题翻译成本体推理问题的手法，是一个可迁移的招。

它要算的是「查询的哪一段可以在某个概念长出来的匿名树里走完」，这看起来是个自动机可达性问题。它的做法是把每个自动机状态造成一个新概念名、把每条转移造成一条概念包含公理，然后**交给现成的推理器判定一个包含关系是否成立**。

这一招的价值在于：不自己写搜索，而是把问题搬到一个已经有成熟求解器的形式里去。同类的做法在别处很常见（把调度问题编码成 SAT、把类型检查编码成约束求解），但能认出「我这个问题其实是那问题」需要刻意去找。记下来作为一个思考模式。

最后说这篇该保持怀疑的地方。

第一，实验的查询是合成的，而且不返回答案。 100 条查询里 62 条连幂集都算不出来，剩下的又有 56 条超时。在这种条件下，「改写在 250 毫秒内完成」这个漂亮数字覆盖的样本相当有限。手写查询只有 5 条，还有 3 条超时。这篇论文的实用性主张，证据强度是「原型能跑通」，不是「可以投产」——论文自己的措辞也很谨慎：「表明用本体知识查询图数据可能终于触手可及」。

第二，OWL 2 标准目前不支持角色包含里的数据测试，所以这篇自己定义的那个能力（左侧属性值测试）里有一部分在评测里根本没被用上。这是标准跟不上论文的情况，不是论文的错，但意味着这个新增能力还没有被真实检验。

第三，只在 Neo4j 上跑过。 论文选 Neo4j 的理由是「它的 GQL 兼容程度足够支撑本次评测」，并且明说打算等真正支持 GQL 的系统出现后再做实验。所以现在还不能说这套东西在 GQL 上成立——它是在 Cypher 上成立的。

一句话版本：这篇把「本体知识编译进图查询」这件卡了十多年的事第一次做成了能跑的原型，卡点是 GQL 标准化解掉的；它同时诚实地暴露了这条路的固有失衡——改写只要 250 毫秒，而改写出来的查询半数以上执行超时。

文中金句

"The proposed algorithms are so unamenable to implementation that, more than a decade later, not a single practical implementation has been proposed." 所提出的算法如此不适合实现，以致十多年过去，一个实用实现都没有出现过。

"The arrival of GQL and SQL/PGQ has removed both obstacles at once." GQL 与 SQL/PGQ 的到来一次性移除了两个障碍。

"the ontology reasoning is compiled entirely into the rewritten query, which can be evaluated with standard query engines without any data preprocessing" 本体推理被完全编译进改写后的查询，它可以由标准查询引擎执行，不需要任何数据预处理。

"Although the database remains agnostic to the ontology, the answer to the rewritten query is complete with respect to the ontology semantics." 尽管数据库对本体一无所知，改写后查询的答案相对本体语义是完备的。

"We expect that real-world queries that require different regular paths to travel separately and join somewhere in the unnamed part of the canonical model will very rarely emerge in practice, if at all." 我们预期，那些需要不同正则路径分别行进、然后在典范模型无名部分某处汇合的真实查询，实践中极少出现，甚至根本不会出现。

"optimising the evaluation of the generated queries was out of scope. From these results, it is clear that there is a need to understand exactly why rewritten queries time-out" 优化生成查询的执行不在本文范围内。从这些结果可以清楚看到，需要弄清究竟为什么改写后的查询会超时。

全文翻译

摘要

Ontology-based data access is intended for graph data, but practical support remains limited to SQL-like languages lacking the navigational and path-matching features fundamental to graph querying.

基于本体的数据访问本来是为图数据设计的，但实际的支持一直局限在类 SQL 的语言上，而那些语言缺少图查询最根本的导航与路径匹配能力。

Theoretical algorithms for navigational queries have long been available. Still, they have never been implemented, largely because practical graph query languages fell short of their theoretical counterparts and lacked the expressive power needed to support rewriting common ontology languages.

导航查询的理论算法早就有了。但它们从未被实现过，主要因为实用的图查询语言够不上它们的理论对应物，缺少支撑常见本体语言改写所需的表达力。

The recent standardisation efforts around GQL and SQL/PGQ finally allow us to overcome this barrier and present a practical query rewriting technique for ontology-mediated navigational graph queries.

围绕 GQL 与 SQL/PGQ 的近期标准化工作终于让我们能越过这道障碍，给出一套面向本体中介导航图查询的实用改写技术。

Ontologies are written in a DL-Lite variant tuned to property graphs, and the queries in a GQL fragment with nested two-way regular path queries and which can be evaluated in Cypher. Preliminary experiments with our proof-of-concept prototype suggest that querying graph data with ontological knowledge may finally be within reach.

本体用一个**针对属性图调过的 DL-Lite 变体**书写，查询用一个带嵌套双向正则路径查询、且可以在 Cypher 里执行的 GQL 片段书写。概念验证原型的初步实验表明，**用本体知识查询图数据可能终于触手可及**。

一、引言

框架的目标。

A core aim of the ontology-based data access (OBDA) framework, in addition to the virtual integration of data sources, is to extend existing data with domain-specific knowledge without the need to materialise all implied facts; the framework still enables access to all consequences when answering user queries.

基于本体的数据访问这个框架，除了虚拟地整合多个数据源之外，一个核心目标是**用领域知识扩展已有数据，而不需要把所有被蕴含的事实都物化出来**；这个框架在回答用户查询时仍然能访问到全部推论。

核心问题与主要技术。

The key question at the core of OBDA is the following: how to capture all the semantic consequences of a query in the presence of an ontology, where the query may be specified over a possibly extended signature with terms from both the data and the ontology.

它的核心问题是：**在有本体存在的情况下，如何捕获一个查询的全部语义推论**——而这个查询可能是在一个扩展了的记号集上写的，里面既有来自数据的词，也有来自本体的词。

The main technique for OMQA is called query rewriting: the query is rewritten into a new query that uses only the reduced signature present in the data, incorporating the knowledge in the ontology into the query itself.

主要技术叫**查询改写**：把查询改写成一个新查询，**新查询只用数据里实际存在的那个较小的记号集，而把本体里的知识并进查询自身**。

The rewritten query can then be evaluated using a conventional database system that does not support ontologies, while still making effective use of the ontology to retrieve implicit answers.

改写后的查询就可以交给一个**并不支持本体的常规数据库系统**去执行，同时仍然有效利用了本体来取回那些隐含的答案。

图数据库这边的空缺。

While commercial OBDA systems already exist for relational databases, corresponding solutions for graph databases — the focus of this paper — are still missing.

关系数据库上**已经有商用的 OBDA 系统**，而图数据库上对应的方案——本文的焦点——**仍然缺失**。

图数据库用的是属性图模型：节点，以及节点之间的二元有向边；节点和边都可以被赋予**多个标签**来分类，也都可以通过所谓**属性**关联到数据值。

The increasing popularity of graph databases has recently culminated in the ISO standardisation of two new graph query languages: GQL and SQL/PGQ. The key distinguishing feature of these query languages is their navigational capability, which allows them to match arbitrarily long paths in the data.

图数据库日渐流行，最近达到高潮的标志是两种新图查询语言的 ISO 标准化：GQL 与 SQL/PGQ。**这两种语言最关键的区别性特征是它们的导航能力——能匹配数据中任意长的路径。**

两者共享同一个导航内核，其中包含标准的导航查询语言：（双向）正则路径查询、嵌套双向正则路径查询，以及它们的合取变体。

过去十年的研究为什么用不上。

However, these studies focused on the boundaries of decidability and computational complexity. The proposed algorithms are so unamenable to implementation that, more than a decade later, not a single practical implementation has been proposed.

然而这些研究关注的是可判定性与计算复杂度的边界。**所提出的算法如此不适合实现，以致十多年过去，一个实用实现都没有出现过。**

We differ from these early theoretical works, by looking to develop practical techniques for pure rewritings of navigational queries (i.e., the ontology reasoning is compiled entirely into the rewritten query, which can be evaluated with standard query engines without any data preprocessing).

我们与这些早期理论工作的不同在于，我们要发展的是导航查询**纯改写**的实用技术——**本体推理被完全编译进改写后的查询**，而它可以由标准查询引擎执行、不需要任何数据预处理。

两个历史障碍。

There were two significant roadblocks that discouraged the development of practical techniques for OMQA over graph databases.

有**两个重大路障**打消了在图数据库上做实用技术的念头。

First, practical languages imposed ad-hoc restrictions on navigational languages that rendered them unsuitable for this setting, for example, by providing only partial support for RPQs and by failing to support the basic homomorphism semantics that is natural in the OMQA context.

第一，实用语言对导航语言施加了**临时性的限制**，让它们不适合这个场景——比如只部分支持正则路径查询，以及不支持本体中介查询语境下天然的基本同态语义。

Second, it has been shown that even plain RPQs in the presence of the simplest DL-Lite ontologies cannot be rewritten into C2RPQs. Instead, such rewritings require nested RPQs, which existing technologies did not support.

第二，已经被证明：**哪怕是最简单的 DL-Lite 本体，普通的正则路径查询也改写不成合取双向正则路径查询**。这类改写需要**嵌套**的正则路径查询，而已有技术不支持嵌套。

The arrival of GQL and SQL/PGQ has removed both obstacles at once: the standards include CN2RPQs and basic homomorphism semantics, and practical graph query languages are now rapidly being updated to support these features. This has finally made practical OMQA over graph databases feasible.

GQL 与 SQL/PGQ 的到来一次性移除了这两个障碍：标准里包含了合取嵌套双向正则路径查询与基本同态语义，而实用图查询语言正在快速更新以支持这些特性。**这终于让图数据库上的实用本体中介查询成为可能。**

架构与三条贡献。

架构由三部分组成：一份用 DL-Lite 表达的本体（例如写成 OWL）；改写算法 **ARCO**；以及一个支持 GQL 的属性图数据库，如 Neo4j、Google Spanner，或通过 SQL/PGQ 的 Oracle Database。用户在扩展记号集上提出查询（**查询里可以同时用本体的词和底层数据库的词**），算法把它编译成一个语义等价的改写查询，其中直接并入了本体推理的后果。

Although the database remains agnostic to the ontology, the answer to the rewritten query is complete with respect to the ontology semantics.

尽管数据库对本体一无所知，改写后查询的答案相对本体语义是完备的。

三条贡献：

(1) We present the first practical algorithm to rewrite navigational ontology-mediated queries with unrestricted regular path expressions, and with DL-Lite ontologies, into graph queries that can be evaluated with off-the-shelf query engines for graph data adopting the property graph data model. The queries can access the data values of properties, also while navigating paths.

第一个**实用**算法，把带**无限制正则路径表达式**、在 DL-Lite 本体下的导航本体中介查询，改写成可以用现成图查询引擎（属性图模型）执行的图查询。**查询可以访问属性的数据值，在沿路径导航的过程中也可以。**

(2) We leverage the power of graph queries with nested path navigation to support the interaction between the regular path expressions in the given queries on the one hand, and the ontology axioms on the other. To our knowledge, this is the first algorithm for a navigational query language that fully supports property paths, which produces a pure rewriting.

利用图查询的**嵌套路径导航**能力，来支撑查询中的正则路径表达式与本体公理之间的相互作用。据作者所知，这是第一个针对完全支持属性路径的导航查询语言、并产出纯改写的算法。

(3) 实现了原型：输入 OWL 本体加导航查询，改写成 Cypher，在 Neo4j 上执行。

关于 join-on-free 的限制：为保证算法保持实用且有用，施加一个叫 **join-on-free** 的限制——非答案变量不能被多个原子共用。完整的合取双向正则路径查询改写技术是已知的，这个片段作为起点。

关于本体侧的扩展：定义一个 DL-Lite 变体，允许在公理左侧使用属性值测试，从而能基于这些值创造概念与角色。

二、改写算法：两个跳过

（这一节把论文第 3、4 节的机制以直觉形式给出，符号定义与证明见原文。）

典范模型。对每一个与本体一致的属性图，存在一个**典范模型**，它恰好给出所有导航查询的确定答案。它由七条规则反复应用而成——概念包含、角色包含、以及右侧存在量词引发的新节点创造。

一个便于理解的说法是把典范模型看成一堆**事实**。在库里出现的叫**显式事实**，其余叫**隐式事实**。每一条隐式事实都由另一条事实触发，顺链子往回追必然追到一条显式事实。

匿名节点与匿名部分。为满足右侧存在量词而新造的节点叫**匿名节点**，提到至少一个匿名节点的那些事实构成典范模型的**匿名部分**。

It is not hard to see that if there is a single fact $C(a)$ in A , then all the anonymous nodes added during the model construction form a tree-shaped graph rooted at a . Moreover, since all rules are deterministic, this tree is unique up to renaming of nodes.

不难看出，如果库里有单独一条事实 $C(a)$ ，那么模型构造过程中加进来的**所有匿名节点构成一棵以 a 为根的树**。而且，因为所有规则都是确定性的，这棵树在节点改名之内是唯一的。

两阶段改写。

Our rewriting of a single N2RPQ q is based on two skipping functions that take an N2RPE as input and output a modified N2RPE with additional transitions that allow it to 'skip over' implicit facts, and instead look for the explicit facts that trigger their generation.

对单个嵌套双向正则路径查询的改写基于**两个跳过函数**：它们输入一个路径表达式，输出一个添加了额外转移的修改版，使它能「跳过」隐式事实，转而去寻找触发这些隐式事实生成的显式事实。

第一阶段 **skipAnon** 添加转移以跳过匿名部分，使得查询在只涉及非匿名个体的事实上的答案，与原查询在整个典范模型上的答案一致。第二阶段 **skipCore** 再添加转移，使得**只在库本身上执行、不带任何隐式事实**时也能得到完备答案。

第一阶段依赖的关键判定，被归约成描述逻辑推理。论文把查询自动机的每个状态换成一个新造的概念名，把自动机的转移换成概念包含公理，于是「查询的这一段能不能在概念 C 长出来的树里走完」这个问题，等

价于一个概念包含关系在改造后的本体里是否成立——可以直接交给现成的推理器判定。

第二阶段的直觉：每次规则应用添加一条转移，允许自动机「跳过」某条隐式事实，改用触发链上更靠前的某条事实。

ARCO 的第一步是消掉非答案变量，方法是用嵌套表达式把它们包起来（论文证明这个变换保持答案不变），然后对每个原子分别应用两个跳过函数。整体的可靠性与完备性有定理保证。

对 join-on-free 的辩护。

To achieve a simple and practicable solution we focused on join-on-free CN2RPQs. This does not seem too limiting: even plain N2RPEs can already express many realistic examples that involve complex bidirectional navigation on the anonymous implicit facts.

为了得到一个简单可行的方案，我们聚焦 join-on-free 的合取嵌套双向正则路径查询。**这看起来限制不算太大：**即便是普通的嵌套双向正则路径表达式，也已经能表达许多涉及在匿名隐式事实上做复杂双向导航的现实例子。

We stress that algorithms for full extended CN2RPQs in expressive DLs have been available for over a decade. It is not difficult to adapt these algorithms and extend our technique to be complete for all CN2RPQs; in fact, we do that in the extended version of this paper. However, this seems to dramatically increase the run time of the rewriting algorithm, and the added value seems limited, as we have not yet found a good use case for queries beyond our join-on-free fragment.

我们强调，表达力更强的描述逻辑上完整合取查询的算法十多年前就有了。把它们改编、把我们的技术扩展到对所有合取嵌套双向正则路径查询完备并不困难——**事实上我们在本文的扩展版里做了这件事。然而这似乎会让改写算法的运行时间急剧上升，而增加的价值看起来有限，因为我们还没有为超出 join-on-free 片段的查询找到一个好用例。**

三、实现与实验

实现。原型用 Java 写：用 OWL API 解析并扩展本体，用 **Hermit** 推理器验证候选，用 ANTLR 解析输入查询，用 JgraphT 做自动机的内部表示，最后用 Cypher DSL 把输出转成 Cypher 语法。源码已匿名共享，将来会公开。

环境。虚拟机 400 GB 内存、16 核，Debian 12；宿主机 AMD EPYC 7313 @ 2.00 GHz；Neo4j 5.25.0 部署在同一台虚拟机上。

数据。两份特征迥异的真实本体：**DBPedia**（从维基百科条目抽取的数据，之所以选它是因为**规模**）；**MMM**（蒙彼利埃都会区的开放数据，覆盖该市下水道系统，之所以选它是因为**这份本体含有右侧存在量词的公理，会触发 skipAnon**）。规模见前表。

选 Neo4j 的理由：它提供的 GQL 兼容程度足以支撑本次实验评测。本体里的概念对应到数据库的节点标签，角色对应到关系类型（即边标签）。

To the best of our knowledge, the current version of the OWL2 standard does not support data tests in role inclusions ... therefore, we do not consider this kind of axioms in the evaluation.

据我们所知，当前版本的 OWL 2 标准不支持角色包含中的数据测试……因此评测中不考虑这类公理。

查询。 由于不存在针对这类改写的基准查询集，采用两种办法：一，为评测改写算法，从本体里随机选概念与角色、随机应用连接/并/克莱尼星等算子，合成 100 条查询，同时施加路径长度与嵌套层数的结构约束；二，为在 Neo4j 上评测改写结果，另外手写 5 条基于 MMM 数据集及其中 SPARQL 查询的查询。

改写耗时的结果。

Although the DBPedia ontology contains significantly more concepts and roles, rewriting is generally faster (under 250 ms) than with the MMM ontology (around 600 ms). This is because DBPedia does not contain any axioms with existentials on the right-hand side.

尽管 DBPedia 本体的概念与角色多得多，它的改写一般更快（250 毫秒以内），而 MMM 约 600 毫秒。原因是 DBPedia 不含任何右侧带存在量词的公理，因此没有相关的跳过元组要算。

从结果可以推断：若存在这类元组，它们对改写时间的影响似乎大于概念数量之多。

It should be noted that for 62 of the 100 generated queries computing the power set was not feasible given the number of states of the query (≥ 30).

应当指出，100 条合成查询里有 62 条，因其状态数 (≥ 30) 导致幂集计算不可行。

执行耗时的结果。

While the evaluation of rewriting times already shows practical feasibility, the generated queries with Neo4j still seems to be a challenge.

改写时间的评测已经显示出实际可行性，但在 Neo4j 上执行这些生成的查询仍然是个挑战。

DBPedia 上查询分成两组：要么在 2 到 8 秒内跑完，要么很快超时（上限 20 秒，100 条中 56 条超时）；与输出转移数只有弱相关。MMM 上手写的 5 条里 3 条超时（上限 1000 秒）。

The reasons for the high evaluation times may be, on the one hand, query constructions that initiate the computation of the Cartesian product or, on the other hand, a high number of query answers. Note, that the generated queries do not return any answers, as could be anticipated. However, optimising the evaluation of the generated queries was out of scope.

执行时间过高的原因，一方面可能是查询构造触发了笛卡尔积的计算，另一方面可能是查询答案数量太多。**注意，正如可以预料的，那些合成查询并不返回任何答案。** 不过，优化生成查询的执行不在本文范围内。

From these results, it is clear that there is a need to understand exactly why rewritten queries time-out and how to achieve optimal rewritten queries.

从这些结果可以清楚看到，需要弄清究竟为什么改写后的查询会超时，以及怎样得到最优的改写查询。

四、结论与未来工作

In this paper, we present the first practical algorithm for rewriting N2RPQs and a significant subset of CN2RPQs, making use of the ability of modern graph databases to express complex navigational queries with nesting.

本文给出[第一个实用算法](#)，改写嵌套双向正则路径查询以及合取版本的一个可观子集，用到了现代图数据库表达带嵌套的复杂导航查询的能力。

We particularly note the ability of our approach to access key values within path expressions in the presence of an ontology with data tests.

我们特别指出本方法的一项能力：[在本体带数据测试的情况下，仍能在路径表达式内部访问键值。](#)

计划的扩展方向：更好地支持属性值（沿用已有的带数据值改写工作）；更具表达力的本体语言（沿用近期识别出的、具有 NL 数据复杂度的更丰富本体语言的工作）。

This work brings us significantly closer to making ontology-based data-access over graph databases a reality, and as such, we aim to explore the potential of our technique for real-life OBDA systems with navigational capabilities, and to conduct experiments on systems supporting GQL as soon as they become available.

这项工作让图数据库上的基于本体的数据访问显著接近现实。[我们打算探索这套技术在具备导航能力的真实 OBDA 系统里的潜力，并在支持 GQL 的系统一出现就在其上做实验。](#)

Another interesting line of future work would be the creation of useful benchmarks that combine both complex navigational queries with rich ontologies expressed over them.

另一条有意思的未来工作是[构造有用的基准](#)，把复杂导航查询与在其上表达的丰富本体结合起来。

资助方：奥地利科学基金（FWF）。

参考链接

- 论文 [arXiv 2608.20092](#) — CIKM 2026, 共 16 页
- [原型源码（匿名）](#) — OWL 转 Cypher 的改写原型, Java
- [GQL 标准](#) — ISO/IEC 39075, 图查询语言标准, 本文的前提条件
- [Neo4j](#) — 实验用的属性图数据库, 选它的理由是 GQL 兼容度够用

- [DBPedia](#) — 实验用的大规模本体与数据集